Before the Search Begins: Generating Function-Informed Initialization for Tight Constrained Binary Optimization

Muhammad Marzouk Baig
MS Candidate, Khoury College of Computer Sciences
Northeastern University, Portland, Maine
baig.muham@northeastern.edu

Abstract

We present a generating function-informed initialization framework for genetic algorithms (GAs) applied to the Multidimensional Knapsack Problem (MKP). Standard initialization—setting each bit to 1 with probability 0.5 independently—ignores the feasibility structure of the search space entirely. On tight instances this produces initial populations with zero feasible solutions, preventing any search from proceeding under constraint-respecting fitness models. The framework is grounded in the combinatorial generating function tradition, applied here to constraint feasibility rather than partition enumeration: for each item j, we derive a sampling probability p_j from sub-generating functions that measure how much feasible space is lost when item j is included versus excluded. The framework is purely constraint-driven: it requires no objective function, no LP relaxation, and no prior search. A GPU-accelerated Monte Carlo approximation provides estimates accurate to within 0.03% in 5–8 seconds. On tight instances ($\alpha = 0.25$), where uniform initialization produces zero feasible solutions, our method achieves 49.8% initial feasibility—versus 6.4% for the best prior method [6] and 0% for uniform—and converges to 97.0% of the reference best under a death penalty model while both baselines fail entirely. Under soft penalty, GF-biased initialization reaches 90% of the reference best 17% faster than uniform at $\alpha = 0.25$. A quality-filtered seeding variant—which screens $k \cdot P$ biased candidates by objective value at negligible additional cost—reduces evaluations to Conv-90 by a further 23% and provides the largest quality advantage at low offspring budgets (+17.7pp over uniform at 200 offspring). The framework degenerates automatically to uniform initialization on loose instances.

1 Introduction

1.1 Motivation

Determining the population initialization approach is the first decision made when using a genetic algorithm on a binary optimization problem, yet it is too often treated as a formality. The standard approach—setting each bit independently to a value of 1 with probability 0.5—is convenient and assumption-free, but this approach ignores the structure of the problem being solved. For unconstrained or loosely constrained problems this is harmless and the resulting random population is largely feasible. For tightly constrained combinatorial problems, it can be catastrophic as there is the chance of creating a completely infeasible starting population.

The generating function connection. Generating functions are a classical tool in combinatorics for encoding subset enumeration in compact algebraic form. The idea goes back at least to Euler and reached remarkable depth in the work of Hardy and Ramanujan [9] on integer partitions: the generating function $\prod_{k=1}^{\infty}(1-x^k)^{-1}$ encodes, in the coefficient of x^n, the number of ways to write n as a sum of positive integers. The same algebraic principle applies to our setting. For items with integer weights $w_1, \ldots, w_n$, the product $\prod_j(1+x^{w_j})$ encodes, in the coefficient of x^k, the number of subsets with total weight exactly k. This maps naturally onto the feasibility question for constrained binary optimization: by factoring out one item at a time and evaluating the remainder against each capacity threshold, we can ask—before any search begins—how much of the feasible space survives when item j is included versus excluded. The closest prior work [6, 7] proposed using a single global inclusion probability derived from a greedy heuristic, applied identically to all items. Our approach instead derives per-item probabilities from sub-generating functions, requires no objective function, and generalizes to any number of constraints. To our knowledge, the generating function connection has not previously been exploited for this purpose. The framework we develop yields a principled, interpretable, and provably self-calibrating initialization method.

1.2 Contributions

The specific contributions of this paper are:

1. A mathematical framework that uses subgenerating functions over order-1 schemas to derive per-item feasibility densities ρ_j^1 and ρ_j^0 for any binary constrained optimization instance (Section 3).
2. A sampling rule $p_j = \rho_j^1/(\rho_j^1 + \rho_j^0)$ that recovers uniform initialization when no constraint structure is present ($p_j = 0.5$) and biases away from infeasibility-inducing items in proportion to the feasibility cost they impose (Section 3).
3. A GPU-accelerated Monte Carlo approximation algorithm that estimates ρ_j^1 and ρ_j^0 to within 0.03% in 5–8 seconds for $n = 100$, $m = 3$, running in $O(n \cdot M \cdot m)$ time with $M = 10^7$ samples (Section 4).
4. A direct empirical comparison against Hill's [6] global-density initialization method across five tightness levels, isolating what per-item differentiation contributes beyond knowing the right global density (Section 6.4).
5. Empirical results showing that GF-biased initialization enables search under a death penalty model where both uniform and Hill initialization fail entirely, and reaches convergence thresholds faster under soft penalty across all tested tightness levels (Section 6).

1.3 Paper Organization

Section 2 reviews the MKP, the Chu–Beasley GA, schema theory, and generating functions. Section 3 develops the mathematical framework. Section 4 presents the exact and Monte Carlo algorithms along with a complexity analysis. Section 5 provides worked examples on small instances. Section 6 presents experiments and results. Section 7 discusses limitations and the broader applicability of the framework. Section 8 concludes.

2 Background

2.1 The Multidimensional Knapsack Problem

The MKP generalizes the classical 0-1 knapsack problem to m constraints. Given n items each with value v_j and weight w_{ji} for constraint i, and m capacity constraints W_i, the MKP is:

$$\text{maximize} \quad \sum_{j=1}^{n} v_j x_j$$

$$\text{subject to} \quad \sum_{j=1}^{n} w_{ji} x_j \le W_i \quad \forall i = 1, \ldots, m$$

$$x_j \in \{0, 1\} \quad \forall j = 1, \ldots, n$$

The MKP is NP-hard and is a canonical benchmark for metaheuristics. Instances are commonly parameterized using the *tightness ratio* $\alpha = W_i / \sum_j w_{ji}$, which controls the fraction of the total item weight that fits in each constraint. Low values of α are considered tight, larger values are considered loose. Standard benchmark instances are available from the OR-Library [2].

2.2 The Problem with Random Generation

Consider the MKP with a tightness ratio $\alpha = W_i / \sum_j w_{ji}$ of 0.25: each capacity is set to 25% of the total item weight across that constraint. On such instances the fraction of all binary strings that are feasible is tiny. Uniform random initialization samples from the full $\{0, 1\}^n$ space without any awareness of where feasible solutions lie; in practice, every solution in an initial population of 50 is infeasible. Under a death penalty fitness model—which assigns zero fitness to infeasible solutions—the entire initial population is non-competing and selection cannot operate, and thus the GA will fail to work well. This failure mode is not an artifact of a poorly chosen penalty function or an implementation bug. It is a structural consequence of initializing without any consideration into the quality of the initial population.

2.3 Chu and Beasley Genetic Algorithm

Chu and Beasley [1] proposed a steady-state GA for the MKP that remains a standard baseline. Key components:

- **Initialization:** Each bit x_j set to 1 with probability 0.5 independently.
- **Crossover:** Uniform crossover.
- **Mutation:** Bit-flip at rate $1/n$.
- **Selection:** Binary tournament.
- **Repair:** A deterministic operator that iteratively drops items (in ascending order of efficiency u_j) until feasibility is restored, then greedily adds items (in descending order of u_j) while feasibility is maintained, where:

$$u_j = \frac{v_j}{\sum_{i=1}^{m} w_{ji}/W_i}$$

The repair operator is effective on standard benchmarks because item efficiency is a reliable proxy for solution quality when values and weights are uncorrelated. It is undefined when no objective exists and potentially misleading when value-weight correlation is adversarial [5].

Because the repair ranking is fixed, diverse infeasible offspring tend to be repaired toward similar feasible solutions, which can contribute to premature convergence on harder instances.

2.4 Hill's Global-Density Initialization

Hill [6] proposed the first problem-informed initialization method for MKP GAs. The approach runs the Toyoda greedy heuristic to find a feasible solution, then uses the ratio of active variables ($x_j = 1$) in that solution as a single global $\Pr(X = 1)$, applied identically to all items. Hill and Hiremath [7] subsequently demonstrated that this global-density method accelerates GA convergence on two-dimensional knapsack problems, particularly on tight instances where the default $\Pr(X = 1) = 0.5$ produces few feasible solutions. Our work replaces the single global probability with per-item probabilities derived from generating functions, eliminating the dependence on objective values and generalizing to any number of constraints.

2.5 Schema Theory

Schema theory, introduced by Holland [3], provides a framework for analyzing GA behavior at the level of partial solutions. A *schema* is a string of length n over $\{0, 1, *\}$, where $*$ is a wildcard matching either value. An *order-k* schema has k fixed positions. The Schema Theorem states that short, low-order, above-average schemata are sampled exponentially more frequently across generations. Initialization determines the initial representation frequency of all schemata; biasing toward high-feasibility order-1 schemata directly seeds the population with the building blocks from which feasible high-quality solutions will be composed.

2.6 Generating Functions in Combinatorics

A generating function is a formal power series whose coefficients encode combinatorial quantities. The tradition is ancient: Euler used generating functions to analyze integer partitions, and the field reached a landmark in the Hardy–Ramanujan asymptotic formula [9] for the partition function $p(n)$, derived by analyzing $\prod_{k=1}^{\infty}(1-x^k)^{-1}$. For a finite set of items with integer weights $w_1, \ldots, w_n$, the analogous product is:

$$f(x) = \prod_{j=1}^{n}(1 + x^{w_j})$$

The coefficient of x^k in $f(x)$ equals the number of subsets with total weight exactly k. This follows from the distributive law: each factor contributes either 1 (item excluded) or x^{w_j} (item included), so the product enumerates all 2^n subsets. Wilf [4] provides a comprehensive treatment of generating functions in combinatorics; the specific application to constraint feasibility and GA initialization developed here is, to our knowledge, new.

3 Mathematical Framework

3.1 Feasibility and the Full Generating Function

Consider the MKP with n items and m constraints. A solution $\mathbf{x} \in \{0, 1\}^n$ is *feasible* if:

$$\sum_{j=1}^{n} w_{ji} x_j \leq W_i \quad \forall i = 1, \ldots, m$$

For m constraints, the full generating function over all 2^n subsets is:

$$f(x_1, \ldots, x_m) = \prod_{j=1}^{n} \left(1 + x_1^{w_{j1}} x_2^{w_{j2}} \cdots x_m^{w_{jm}}\right)$$

A term $x_1^{k_1} \cdots x_m^{k_m}$ in the expanded product corresponds to a feasible subset if and only if $k_i \leq W_i$ for all i.

3.2 Sub-Generating Functions and Feasibility Densities

To analyze feasibility at the schema level, we define the *sub-generating function* for item j as the generating function over all $n - 1$ remaining items:

$$g_j(x_1, \ldots, x_m) = \prod_{k \neq j} \left(1 + x_1^{w_{k1}} \cdots x_m^{w_{km}}\right)$$

This function enumerates all 2^{n-1} subsets of items other than j. The function g_j is the same regardless of whether item j is fixed to 0 or 1; the distinction lies solely in the capacity threshold applied when counting feasible terms:

- When item j is **included** ($x_j = 1$), its weight consumes capacity, leaving $W_i - w_{ji}$ for the remaining items. The feasibility density is:

$$\rho_j^1 = \frac{\#\{\text{terms in } g_j : \text{exponent of } x_i \leq W_i - w_{ji}\ \forall i\}}{2^{n-1}}$$

- When item j is **excluded** ($x_j = 0$), full capacity is available. The feasibility density is:

$$\rho_j^0 = \frac{\#\{\text{terms in } g_j : \text{exponent of } x_i \leq W_i\ \forall i\}}{2^{n-1}}$$

Intuitively, ρ_j^1 is the probability that a uniformly random solution *containing* item j is feasible; ρ_j^0 is the same for solutions *not* containing j.

3.3 Order-1 Schemas and the Schema-Level Interpretation

A schema is a string over $\{0, 1, *\}$ where $*$ is a wildcard. An order-1 schema has exactly one fixed position. For n items there are $2n$ order-1 schemas—two per item—each containing exactly 2^{n-1} strings. The

sub-generating function g_j corresponds exactly to the order-1 schema for item j: it enumerates and characterizes the feasibility of all 2^{n-1} strings that agree with item j's fixed value. Restricting the analysis to order-1 structures requires n sub-generating function computations (one per item) rather than the 3^n required for full schema enumeration.

3.4 Sampling Probability

The initialization probability for bit j is allocated proportionally to the feasibility signal of each choice. For two mutually exclusive options with scores a and b, the proportional allocation rule assigns probability $a/(a+b)$ to option a. Applied to the two feasibility densities:

$$p_j = \frac{\rho_j^1}{\rho_j^1 + \rho_j^0}.$$

This rule satisfies two important properties:

1. $p_j + (1 - p_j) = 1$ (the probabilities sum to one by construction).

2. The ratio of probabilities equals the ratio of feasibility densities: if excluding item j is twice as feasibility-preserving as including it, the algorithm is twice as likely to exclude it.

Boundary cases. When $\rho_j^1 = \rho_j^0$, we have $p_j = 0.5$, recovering standard uniform initialization. This occurs on loose instances (α near 1), where the presence or absence of any single item has negligible effect on feasibility. When $\rho_j^1 = 0$, item j is never compatible with any completion of the remaining items—$p_j = 0$ and the item is never selected. When $\rho_j^0 = 0$, all completions are infeasible regardless of whether j is included or not, which cannot occur on any instance for which the empty solution is feasible.

4 Algorithm

4.1 Exact Computation

The exact procedure computes ρ_j^1 and ρ_j^0 by explicitly enumerating all 2^{n-1} terms of each sub-generating function. Note that the full generating function f itself is never expanded; only the n sub-generating functions g_j are required, each of size 2^{n-1}.

Algorithm 1 GF-Informed Initialization (Exact)

Require: n items, m constraints, weights w_{ji}, capacities W_i, population size P
Ensure: Initial population of P solutions
Phase 1 — Precomputation (done once)
for $j = 1$ to n **do**
  Expand $g_j(x_1, \ldots, x_m) = \prod_{k \neq j} (1 + x_1^{w_{k1}} \cdots x_m^{w_{km}})$
  $\rho_j^1 \leftarrow$ fraction of terms where exponent of $x_i \leq W_i - w_{ji}$ for all i
  $\rho_j^0 \leftarrow$ fraction of terms where exponent of $x_i \leq W_i$ for all i
  $p_j \leftarrow \rho_j^1/(\rho_j^1 + \rho_j^0)$
end for
Phase 2 — Population Initialization
for $s = 1$ to P **do**
  for $j = 1$ to n **do**
    $x_j \leftarrow 1$ with probability p_j, else 0
  end for
  Add solution $\mathbf{x}$ to population
end for
return population

Exact computation requires enumerating 2^{n-1} terms per item and is feasible only for $n \leq 20$ in practice. Phase 2 is identical to standard uniform initialization except each bit j is sampled with probability p_j rather than a fixed 0.5.

4.2 Monte Carlo Approximation

For practical instances ($n = 50$–500), exact enumeration is intractable. We approximate ρ_j^1 and ρ_j^0 via Monte Carlo sampling. Instead of enumerating all 2^{n-1} completions, we draw M random binary vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ over the $n-1$ items other than j, and count the fraction that are feasible under each threshold:

$$\hat{\rho}_j^1 = \frac{1}{M} \sum_{k=1}^{M} \mathbf{1}\left[\sum_{\ell \neq j} w_{\ell i}\, s_\ell^{(k)} \leq W_i - w_{ji} \;\; \forall i\right]$$

$$\hat{\rho}_j^0 = \frac{1}{M} \sum_{k=1}^{M} \mathbf{1}\left[\sum_{\ell \neq j} w_{\ell i}\, s_\ell^{(k)} \leq W_i \;\; \forall i\right]$$

where $s_\ell^{(k)} \overset{\text{i.i.d.}}{\sim} \text{Bernoulli}(0.5)$.

Accuracy. Since each $\mathbf{1}[\cdot]$ is a Bernoulli random variable, the standard error of $\hat{\rho}_j^1$ is $\sqrt{\rho_j^1(1-\rho_j^1)/M} \leq 1/(2\sqrt{M})$. At $M = 10^7$, this upper bound is 0.016%—sufficient for initialization purposes.

GPU acceleration. The $M \times n$ matrix of random binary samples can be generated and evaluated entirely on a GPU using PyTorch tensor operations. On a T4 GPU (Google Colab), computing all n probability estimates

for $n = 100$, $m = 3$, $M = 10^7$ takes 5–8 seconds. This is a one-time precomputation before the GA begins.

Failure mode. When $\hat{\rho}_j^1 = 0$ (no Monte Carlo sample is feasible with item j included), $p_j = 0$ and item j is never selected. This is the correct behavior when item j truly blocks all feasible completions, but can be an estimation artifact on very tight instances. In our experiments at $\alpha = 0.25$, some items have estimated $p_j = 0$ (never include) and others $p_j = 1$ (always include), reflecting genuine extreme constraint structure rather than sampling failure.

4.3 Complexity Analysis

Exact computation. Expanding each sub-generating function g_j enumerates 2^{n-1} terms, and checking feasibility against m constraints requires $O(m)$ work per term. Since there are n items, Phase 1 costs $O(n \cdot 2^n \cdot m)$—exponential in n and intractable for $n > 20$ in practice.

Monte Carlo approximation. Each of the n items requires M sample evaluations, each checking m constraints. Phase 1 therefore costs $O(n \cdot M \cdot m)$—linear in n, M, and m, and fully parallelizable across the M sample dimension on a GPU. Concretely, for $n = 100$, $m = 3$, $M = 10^7$, this involves approximately 3×10^9 scalar operations, parallelized to 5–8 seconds on a T4 GPU. The approximation thus reduces an exponential enumeration to a polynomial estimation at the cost of a controllable error bounded by $1/(2\sqrt{M})$.

Phase 2 (population initialization) costs $O(P \cdot n)$ in both cases and is dominated by Phase 1 for any reasonable population size P.

Comparison to LP-based approaches. LP relaxation-based initialization methods require solving an optimization subproblem and depend on an objective function. The GF Monte Carlo approach requires only weights and capacities, solves no optimization problem, and scales linearly in n, m, and M.

Memory. The Monte Carlo approach stores an $M \times (n-1)$ binary matrix in GPU memory. At $M = 10^7$ and $n = 100$, this is approximately 125 MB—manageable on modern hardware and straightforwardly batchable if memory is limited.

5 Worked Examples

5.1 Single Constraint Example

Consider the following MKP instance with $n = 3$ items and $m = 1$ constraint:

	Item 1	Item 2	Item 3
Weight	3	2	4
Value	5	4	3

Table 1. *3-item single constraint instance. Capacity $W = 5$.*

Step 1 — Full Generating Function

$$f(x) = (1 + x^3)(1 + x^2)(1 + x^4)$$
$$= 1 + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^9$$

Of the $2^3 = 8$ subsets, the feasible ones (exponent ≤ 5) are: $\emptyset$, $\{2\}$, $\{1\}$, $\{3\}$, $\{1, 2\}$—giving overall feasibility density $5/8 = 0.625$.

Step 2 — Sub-Generating Functions

Each $g_j(x)$ is the product over the two items other than j and is evaluated against two thresholds: $W - w_j$ (item included) and W (item excluded).

Item 1 ($w_1 = 3$, $W - w_1 = 2$): $g_1(x) = (1+x^2)(1+x^4) = 1 + x^2 + x^4 + x^6$. Of 4 terms: 2 have exponent ≤ 2 and 3 have exponent ≤ 5.

$$\rho_1^1 = 0.50 \quad \rho_1^0 = 0.75 \quad p_1 = 0.40$$

Item 2 ($w_2 = 2$, $W - w_2 = 3$): $g_2(x) = (1+x^3)(1+x^4) = 1 + x^3 + x^4 + x^7$. Of 4 terms: 2 have exponent ≤ 3 and 3 have exponent ≤ 5.

$$\rho_2^1 = 0.50 \quad \rho_2^0 = 0.75 \quad p_2 = 0.40$$

Item 3 ($w_3 = 4$, $W - w_3 = 1$): $g_3(x) = (1+x^3)(1+x^2) = 1 + x^2 + x^3 + x^5$. Of 4 terms: 1 has exponent ≤ 1 and 4 have exponent ≤ 5.

$$\rho_3^1 = 0.25 \quad \rho_3^0 = 1.00 \quad p_3 = 0.20$$

Step 3 — Summary and Expected Feasibility Rate

	Item 1	Item 2	Item 3
Weight	3	2	4
ρ_j^1	0.50	0.50	0.25
ρ_j^0	0.75	0.75	1.00
p_j **(ours)**	**0.40**	**0.40**	**0.20**
p_j (uniform)	0.50	0.50	0.50

Table 2. *Per-item feasibility densities and sampling probabilities for the single-constraint example.*

Item 3 consumes 80% of the capacity upon inclusion; the framework pushes p_3 to 0.20. Enumerating all 8 strings explicitly, the expected feasibility rate under our method is **87.2**% versus 62.5% under uniform initialization—a gain of 24.7 percentage points.

5.2 Multiple Constraint Example

Consider the following MKP instance with $n = 4$ items and $m = 2$ constraints:

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
Value	4	7	5	3

Table 3. *4-item two-constraint instance.* $W_1 = 7$, $W_2 = 8$.

Of the $2^4 = 16$ possible solutions, 9 are feasible (overall density 56.25%). The two-constraint setting requires evaluating each completion against both thresholds simultaneously. We work through Item 4 in full to illustrate how the multi-dimensional threshold check works, then summarize all four items.

Sub-generating function for Item 4 ($w_{41} = 5$, $w_{42} = 1$):

$$g_4(x_1, x_2) = (1 + x_1^2 x_2^3)(1 + x_1^4 x_2^2)(1 + x_1^3 x_2^5)$$

When Item 4 is **included**, the remaining capacity is $W_1 - w_{41} = 2$ and $W_2 - w_{42} = 7$. When Item 4 is **excluded**, the full capacity $W_1 = 7$, $W_2 = 8$ is available. Table 4 enumerates all $2^3 = 8$ completions of g_4.

$\{1,2,3\}$ subset	e_1	e_2	Incl	Excl
$\emptyset$	0	0	✓	✓
$\{3\}$	3	5	✗	✓
$\{2\}$	4	2	✗	✓
$\{2,3\}$	7	7	✗	✓
$\{1\}$	2	3	✓	✓
$\{1,3\}$	5	8	✗	✓
$\{1,2\}$	6	5	✗	✓
$\{1,2,3\}$	9	10	✗	✗

Table 4. *All 8 completions of* g_4. e_1, e_2 *are total weights from items 1–3. Incl:* $e_1 \leq 2$, $e_2 \leq 7$. *Excl:* $e_1 \leq 7$, $e_2 \leq 8$.

From Table 4: 2 of 8 completions are feasible when Item 4 is included; 7 of 8 are feasible when it is excluded.

$$\rho_4^1 = 0.250 \quad \rho_4^0 = 0.875 \quad p_4 = 0.222$$

Item 4 consumes 71% of W_1 on its own, leaving almost no room for any other item on C1 when it is included—which is why p_4 is pushed down to 0.222. Notice that this signal comes specifically from C1: Item 4's weight on C2 is only $w_{42} = 1$, which is harmless. The two-dimensional threshold check automatically identifies which constraint is doing the work.

The same procedure applied to all four items yields:

	Item 1	Item 2	Item 3	Item 4
Weight (C1)	2	4	3	5
Weight (C2)	3	2	5	1
ρ_j^1	0.500	0.375	0.375	0.250
ρ_j^0	0.625	0.750	0.750	0.875
p_j **(ours)**	**0.444**	**0.333**	**0.333**	**0.222**
p_j (uniform)	0.500	0.500	0.500	0.500

Table 5. *Per-item probabilities for the two-constraint example.*

Enumerating all 16 strings with these p_j values, the expected feasibility rate under our method is **83.8**% versus 56.25% under uniform initialization—a gain of 27.6 percentage points.

Note that Items 2 and 3 are penalized equally ($p_j = 0.333$) despite very different weight profiles: Item 2 is heavy on C1 and light on C2, while Item 3 is the reverse. Both are constrained equally by the combination of the two thresholds. This joint accounting—automatically identifying which constraints bind for each item—is what distinguishes the multi-constraint GF approach from heuristics based on single constraints considered independently.

6 Experiments and Results

6.1 Setup

We generate synthetic MKP instances with $n = 100$ items and $m = 3$ constraints using a fixed random seed (42). Item weights w_{ji} and values v_j are drawn uniformly from $[1, 100]$. Capacities are set to $W_i = \alpha \cdot \sum_j w_{ji}$ for $\alpha \in \{0.25, 0.30, 0.35, 0.40, 0.50\}$, giving five instances ranging from tight to medium.

Following Hill's [6] suggestion to level the field beyond the death penalty comparison, we evaluate four initialization and fitness model configurations:

1. **Uniform + soft penalty:** Each bit set to 1 with probability 0.5; fitness $= v(\mathbf{x}) - 50 \cdot$ violation.
2. **Hill (1999) + soft penalty:** Global $\Pr(X = 1)$ from the Toyoda greedy heuristic active-variable fraction, applied identically to all items; same soft penalty.
3. **GF-Biased + death penalty:** Per-item $\hat{p}_j$ via Monte Carlo; infeasible solutions receive fitness 0.
4. **Hybrid 25% GF + soft penalty:** 25% of the initial population drawn from the GF-biased distribution, 75% uniform; soft penalty fitness.

GA parameters:

Parameter	Value
Population size P	50
Offspring per run	20,000
Mutation rate	$1/n = 0.01$
Crossover	Uniform
Selection	Binary tournament ($k = 2$)
Independent runs	20
Monte Carlo samples M	10^7 (2×10^7 at $\alpha = 0.25$)
Soft penalty multiplier	50
Hybrid GF fraction	25%
ILS restarts r	50
ILS perturbation size k	5 (bit-flips)

Table 6. *Experimental parameters. ILS applies only to Section 6.9.*

Reference best. A reference best fitness is established per α value by running a repair-based GA (Chu & Beasley) for 30,000 offspring over 20 runs and taking the best observed value: 2375 ($\alpha = 0.25$); 2739 ($\alpha = 0.30$); 3056 ($\alpha = 0.35$); 3373 ($\alpha = 0.40$); 3909 ($\alpha = 0.50$). All quality figures are expressed as a percentage of this ceiling.

6.2 Initialization Feasibility Under Death Penalty

Table 7 reports initialization feasibility and final solution quality under the death penalty model at $\alpha \in \{0.25, 0.50, 0.75\}$.

α	Method	Init Feas.	Final (% ref)
0.25	Uniform	0.0%	0.0%
0.25	GF-Biased	**49.8%**	**97.0%**
0.50	Uniform	32.6%	98.5%
0.50	GF-Biased	**71.6%**	**98.8%**
0.75	Uniform	100.0%	99.5%
0.75	GF-Biased	100.0%	99.5%

Table 7. *Initialization feasibility and final fitness under a death penalty model. Averages over 20 runs. The $\alpha = 0.75$ rows are identical because $p_j = 0.5$ for all items at this tightness level.*

The result at $\alpha = 0.25$ is the central finding. Under uniform initialization, every solution in every initial population across all 20 runs is infeasible. The death penalty assigns fitness 0 to all solutions, selection cannot differentiate between them, and no improvement ever occurs. GF-biased initialization starts with 49.8% of the initial population feasible, selection operates immediately, and the GA converges to 97.0% of the reference best. This is not a marginal improvement—it is the difference between a completely inoperative algorithm and one that solves the problem effectively. At $\alpha = 0.50$, both methods converge to comparable final quality (98.5% vs. 98.8%), but GF-biased starts with over twice the fraction of feasible solutions (71.6% vs. 32.6%) and converges faster (Section 6.5).

6.3 Sampling Probability Behavior

Table 8 summarizes the distribution of estimated $\hat{p}_j$ values across all items at each tightness level.

α	Min	Max	Mean	Interpretation
0.25	0.000	1.000	0.240	Strong diff. (tight)
0.50	0.414	0.493	0.449	Weak diff. (medium)
0.75	0.500	0.500	0.500	None (loose)

Table 8. *Distribution of $\hat{p}_j$ across all 100 items at representative tightness levels.*

At $\alpha = 0.75$, every item has $\hat{p}_j = 0.5$ by construction: the framework degenerates automatically to uniform initialization. This is a theoretical prediction of Section 3.4 confirmed exactly. At $\alpha = 0.25$, the range $[0, 1]$ indicates items so weight-dominant they are never compatible with any feasible completion ($p_j = 0$), while others are always compatible ($p_j = 1$).

6.4 Gen-0 Feasibility: GF vs. Hill vs. Uniform

Table 9 directly compares generation-zero feasibility rates. Hill's method applies a single global $\Pr(X = 1)$ equally to all items; GF-biased uses a distinct p_j per item.

α	Uniform	Hill	GF	Hybrid	GF−Hill
0.25	0.0%	6.4%	**49.8%**†	12.2%	+43.4pp
0.30	0.0%	6.5%	**50.5%**	12.1%	+44.0pp
0.35	0.0%	7.4%	**54.8%**	13.0%	+47.4pp
0.40	0.7%	7.8%	**58.2%**	15.0%	+50.4pp
0.50	34.3%	6.7%	**71.0%**	40.8%	+64.3pp

Table 9. *Generation-zero feasibility rate (%) averaged over 20 runs ($n = 100$, $m = 3$, seed=42). GF-biased uses death penalty; Uniform, Hill, and Hybrid use soft penalty ($\times 50$). $^\dagger M = 2\times10^7$ at $\alpha = 0.25$. At $\alpha = 0.50$, the Toyoda heuristic produces a conservative global $\Pr(X = 1) \approx 0.56$ below the natural feasible-solution density, which is why Hill underperforms uniform at that level.*

The GF − Hill column isolates what per-item differentiation contributes beyond knowing the right global density. Even at $\alpha = 0.25$ where Hill provides 6.4% feasibility, GF-biased achieves 49.8%—a gap of 43.4 percentage points that cannot be attributed to global density

alone. The hybrid achieves 12.2% at $\alpha = 0.25$, nearly double Hill's 6.4%, by introducing even a minority of GF-informed solutions.

6.5 Convergence Speed

Table 10 reports the median offspring evaluated to reach each fitness threshold at $\alpha \in \{0.25, 0.50\}$ under the death penalty model. The $\alpha = 0.75$ case is omitted as both methods are identical by construction.

α	Method	70%	80%	90%	95%	98%
0.25	Uniform	never (0/20 runs)				
0.25	GF	481	1,023	2,309	7,529	16,845*
0.50	Uniform	82	316	893	1,740	8,327
0.50	GF	54	255	788	1,577	6,724

Table 10. *Median offspring to first reach each fitness threshold (% of reference best) under death penalty. *reached in only 1 of 20 runs within 20,000 offspring.*

GF-biased initialization also yields faster early convergence at $\alpha = 0.50$ across all thresholds, consistent with the higher initial feasibility giving selection an earlier head start.

6.6 Penalty Function Comparison

Table 11 compares all methods under their respective fitness models. Following Hill's [6] observation that penalty functions compensate for poor initialization over time, we include the soft penalty comparison to assess whether the gen-0 feasibility advantage translates to convergence improvements when infeasible solutions are not immediately eliminated.

α	Method	Gen-0	Final	Conv-90
0.25	Uniform + soft	0.0%	97.0%	1,582
0.25	Hill + soft	6.4%	97.5%	**1,171**
0.25	GF + death	49.8%	97.0%	1,345
0.25	Hybrid + soft	12.2%	97.3%	1,488
0.30	Uniform + soft	0.0%	97.3%	1,300
0.30	Hill + soft	6.5%	97.1%	1,435
0.30	GF + death	50.5%	**97.7%**	**1,004**
0.30	Hybrid + soft	12.1%	96.8%	1,038
0.35	Uniform + soft	0.0%	**98.3%**	1,436
0.35	Hill + soft	7.4%	97.7%	1,077
0.35	GF + death	54.8%	97.9%	**936**
0.35	Hybrid + soft	13.0%	98.1%	943
0.40	Uniform + soft	0.7%	97.5%	1,225
0.40	Hill + soft	7.8%	98.0%	990
0.40	GF + death	58.2%	**98.1%**	**917**
0.40	Hybrid + soft	15.0%	97.9%	996
0.50	Uniform + soft	34.3%	98.3%	732
0.50	Hill + soft	6.7%	98.6%	853
0.50	GF + death	71.0%	98.8%	**695**
0.50	Hybrid + soft	40.8%	**98.9%**	727

Table 11. *All methods compared under their respective fitness models. Conv-90 = median offspring to first reach 90% of reference best. Averages over 20 independent runs.*

Under soft penalty, all four methods converge to comparable final quality (97.0–98.9%), consistent with Hill's observation. However, GF-biased initialization reaches Conv-90 fastest at $\alpha \geq 0.30$. At $\alpha = 0.25$, Hill's global-density method reaches Conv-90 in 1,171 offspring—faster than GF-biased under death penalty (1,345)—because the soft penalty provides a gradient that partially substitutes for feasibility awareness when constraints are very tight. The primary advantage of GF-biased initialization is therefore twofold: it enables search entirely in the death penalty regime where both other methods fail, and it provides robust Conv-90 advantages at $\alpha \geq 0.30$.

6.7 Convergence Curves

Figure 1 shows best-so-far convergence curves averaged over 20 runs at $\alpha \in \{0.25, 0.50\}$, zoomed to 5,000 offspring. At $\alpha = 0.25$ (top panel), GF-biased initialization starts highest and converges faster than uniform throughout the early phase. At $\alpha = 0.50$ (bottom panel), GF-biased and hybrid are consistently above uniform and Hill.

6.8 Hybrid Initialization

Table 12 sweeps the GF-biased fraction of the initial population at $\alpha = 0.25$ to identify the minimum informed fraction needed for reliable convergence.

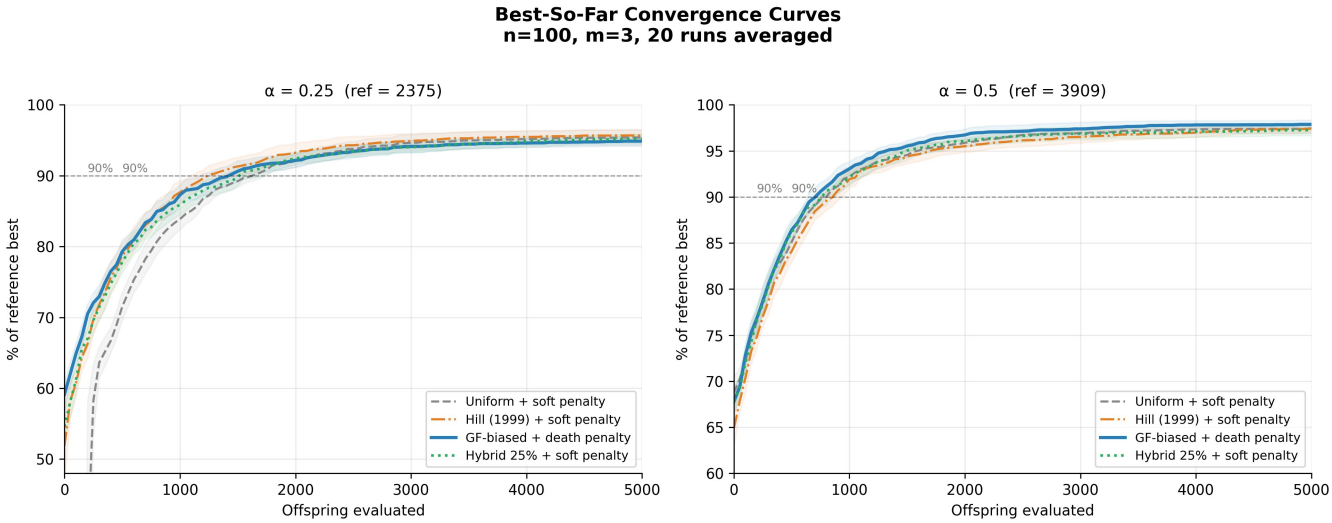

Figure 1. *Best-so-far convergence curves averaged over 20 independent runs ($n = 100$, $m = 3$, seed=42). Shaded bands show ± 0.5 standard deviations.* ***Top:*** *$\alpha = 0.25$ (ref=2375).* ***Bottom:*** *$\alpha = 0.50$ (ref=3909). GF-biased (solid blue) starts highest due to the gen-0 feasibility advantage. All soft-penalty methods converge to comparable final quality; the primary GF advantage is earlier feasibility and faster initial climb.*

Inf. Frac.	Init Feas.	Quality (% ref)	Solved
0%	0.0%	0.0%	0/20
25%	11.6%	97.1%	20/20
50%	25.1%	96.7%	20/20
75%	37.9%	97.1%	20/20
100%	50.5%	96.6%	18/20

Table 12. *Hybrid initialization sweep at $\alpha = 0.25$. "Inf. Frac." is the GF-biased proportion of the initial population; the remainder are uniform. "Solved" counts runs reaching $\geq 95\%$ of the reference best. 20 runs per condition.*

Method	Quality (% ref)	Solved
Uniform + death penalty	0.0%	0/20
GF-Biased	97.0%	20/20
GF-Biased + ILS ($r = 50$, $k = 5$)	**99.3%**	20/20

Table 13. *Solution quality at $\alpha = 0.25$ across pipeline configurations. 20 runs each. "Solved" counts runs reaching $\geq 95\%$ of the reference best. A repair-based GA achieves 99.8% as a non-comparable reference (it uses objective values unavailable to the death-penalty pipeline).*

Seeding just 25% of the population with GF-biased solutions is sufficient to rescue the algorithm: all 20 runs converge to 97%+ quality, compared to 0/20 under pure uniform initialization. Mean solution quality is approximately flat across all non-zero informed fractions (96.6–97.1%), suggesting the critical transition is binary: any non-zero informed fraction crosses the feasibility threshold required for selection to operate. The pure GF condition (100%) solves 18/20 runs rather than 20/20, consistent with lower population diversity when all solutions are drawn from the same biased distribution.

6.9 Full Pipeline at $\alpha = 0.25$

Table 13 shows what is achievable at $\alpha = 0.25$ as downstream optimization is added to the GF-biased baseline. Adding a lightweight ILS step ($r = 50$ restarts, $k = 5$ bit-flip perturbations) to the GF-biased GA brings mean quality to 99.3% of the reference best. The GF framework slots directly into standard local search techniques, and substantial quality gains are available through straightforward downstream optimization.

6.10 Quality-Filtered Seeding

A natural extension of GF-biased initialization is to screen candidate solutions by objective quality before seeding the population. Standard GF-biased initialization generates exactly P solutions using the per-item probabilities p_j and accepts all of them regardless of fitness. Quality-filtered seeding generates $k \cdot P$ candidates (here $k = 8$, yielding 400 candidates for a population of 50), retains only feasible solutions, and seeds with the top P by objective value. The additional cost is negligible: generating 350 extra candidate solutions requires no additional p_j computation and takes under one second.

Table 14 summarizes results at $\alpha = 0.25$.

Method	Penalty	Gen-0 Feas.	Gen-0 Qual.	Final %ref	Conv-90
Uniform	soft×50	0.0%	—	97.0%	1,784
GF-biased	death	49.8%	59.1%	97.0%	1,485
Qual.-filt.	soft×50	100%	64.2%	97.3%	**1,143**
Qual.-filt.	death	100%	64.2%	97.3%	1,306

Table 14. *Quality-filtered seeding at $\alpha = 0.25$, $n = 100$, $m = 3$, seed=42, 20 runs. Gen-0 Qual. = mean fitness of initial feasible solutions as % of reference best. Conv-90 = median offspring to first reach 90% of reference best.*

Under quality-filtered seeding, the initial population is 100% feasible with a mean gen-0 quality of 64.2% of reference best, compared to 59.1% for standard GF-biased and 0% for uniform. Combined with a soft penalty model, quality-filtered seeding reaches 90% of reference best in 1,143 offspring—36% fewer evaluations than uniform (1,784) and 23% fewer than standard GF-biased (1,485). Final solution quality at 20,000 offspring remains comparable across all methods (∼97%), confirming that the primary contribution is early convergence speed rather than asymptotic quality.

Figure 2 shows the convergence curves. The two quality-filtered curves start identically (both 100% feasible, both at 64.2% gen-0 quality) because filtering happens before the GA runs; penalty type only affects what happens after generation zero. The soft penalty variant reaches Conv-90 slightly earlier (1,143 vs. 1,306 offspring) because it can exploit near-infeasible high-value offspring during crossover. The death penalty variant reaches Conv-95 faster (4,129 vs. 4,668 offspring) because it stays strictly within feasible space and avoids fitness regression.

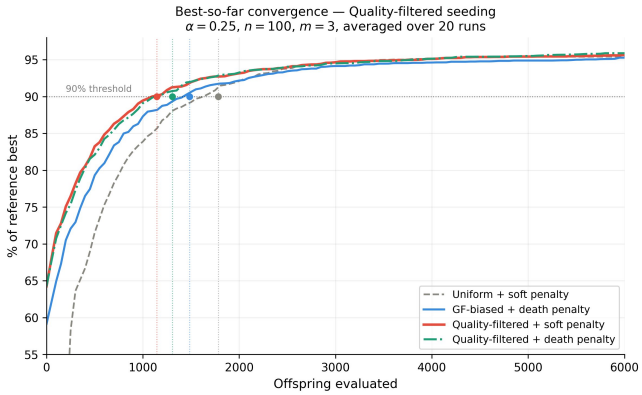

Figure 2. *Best-so-far convergence curves at $\alpha = 0.25$, averaged over 20 independent runs. Vertical dotted lines and filled circles mark Conv-90 for each method. Quality-filtered seeding (red solid, green dash-dot) reaches 90% of the reference best in 1,143 offspring (soft penalty) and 1,306 (death penalty), the fastest of all conditions tested. The two quality-filtered curves are identical at generation zero and diverge only after evolution begins.*

Budget sensitivity

The preceding results use a fixed budget of 20,000 offspring. Table 15 and Figure 3 report final solution quality across offspring budgets, showing that the initialization advantage is largest precisely when budgets are most constrained.

Budget	Uniform +soft	Hill +soft	GF +death	QF +soft	QF +death
200	57.4%	66.4%	70.5%	**75.1%**	74.2%
500	71.5%	78.8%	79.3%	**83.2%**	82.1%
1,000	84.2%	87.7%	87.3%	**89.5%**	89.1%
2,000	92.1%	93.2%	92.2%	93.1%	**93.3%**
5,000	95.4%	**95.7%**	94.9%	95.4%	95.5%
10,000	96.0%	**96.6%**	96.0%	96.5%	96.5%
20,000	97.0%	**97.5%**	97.0%	97.3%	97.3%

Table 15. *Final solution quality (% of reference best) vs. offspring budget at $\alpha = 0.25$, averaged over 20 runs. QF = quality-filtered seeding ($k = 8$).*

At 200 offspring, quality-filtered seeding (75.1%) outperforms uniform by 17.7 percentage points; at 1,000 offspring the gap is still 5.3 points. By 5,000 offspring all methods converge to within 1 point. A user with a budget of 500 offspring gets 83% of reference best with quality-filtered seeding versus 72% with uniform—a gap that would require roughly 3× the budget to close under uniform initialization.

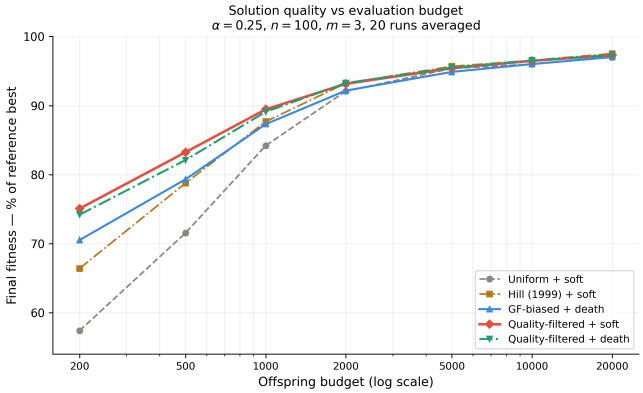

Figure 3. *Solution quality vs. offspring budget at $\alpha = 0.25$ ($n = 100$, $m = 3$, seed=42, 20 runs averaged, log-scale x-axis). The initialization advantage is largest at low budgets and diminishes as all methods converge to ≈97% by 20,000 offspring.*

7 Discussion

7.1 When Does GF-Biased Initialization Help?

The results suggest a clear characterization of when the framework adds value. Two distinct regimes emerge:

Death penalty regime. At $\alpha \leq 0.30$, uniform initialization produces zero feasible solutions and Hill's global-density method produces too few ($\leq 7\%$) to sustain selection pressure under death penalty. GF-biased initialization is the only method that functions. This is the framework's strongest claim and is unambiguous.

Soft penalty regime. Under soft penalty, all methods eventually converge to comparable final quality. GF-biased initialization contributes earlier feasibility and faster convergence to quality thresholds, with Conv-90 advantages at $\alpha \geq 0.30$. At $\alpha = 0.25$, Hill's simpler global-density approach is competitive on Conv-90 because soft penalty provides a gradient that partially substitutes for feasibility awareness. The hybrid (25% GF-biased, 75% uniform) preserves population diversity and is competitive with pure GF-biased on final quality throughout.

When the instance is loose ($\alpha = 0.75$), the framework degenerates to uniform initialization automatically. This graceful degradation is a theoretical property of the p_j formula, not a tuned behavior, and the experiments confirm it.

7.2 Applicability Beyond Standard MKP

The framework's most important property is that it is *objective-free*: p_j depends only on weights w_{ji} and capacities W_i, not on values v_j. This makes it applicable to settings where repair operators are undefined or unreliable:

- **Constraint satisfaction problems (CSPs):** When no objective function exists, the efficiency ranking u_j cannot be computed and repair is undefined. GF-biased initialization provides a principled starting point for any constraint-feasible-only search.
- **Noisy or uncertain objectives:** When value estimates are unreliable, u_j may be misleading. The GF framework ignores objective values entirely and remains well-calibrated.
- **Adversarial correlation structure:** Hill and colleagues [8] showed that standard greedy repair degrades on instances where item weights and values are negatively correlated, because the efficiency ranking systematically disadvantages heavy, high-value items. GF-biased initialization is unaffected by value-weight correlation. Demonstrating this advantage empirically is a primary direction for follow-up work (Section 8.1).

7.3 Computational Overhead

The one-time precomputation at $M = 10^7$, $n = 100$, $m = 3$ runs in 5–8 seconds on a T4 GPU (Section 4.3). For larger-scale applications (e.g., $n = 500$ as in OR-Library benchmarks), the cost scales as $O(n \cdot M \cdot m)$—linearly in n—and is fully GPU-parallelizable across the sample dimension.

Amortization across population size. The precomputation cost is fixed per instance, independent of population size P and offspring budget. The amortized cost per initialized solution is therefore T_{precomp}/P: approximately 0.10–0.16 seconds per solution at $P = 50$, dropping to 0.025–0.04 seconds at $P = 200$. Larger populations make the precomputation relatively cheaper per solution—the opposite of the repair operator, whose cost is incurred once per offspring regardless of population size.

Amortization across runs. If the same instance is evaluated k times—for hyperparameter tuning, multi-seed experiments, or benchmark evaluation—the p_j vector is computed once and reused across all k runs. The amortized precomputation cost per run is T_{precomp}/k, which vanishes as k grows. This is directly relevant to standard benchmark practice: OR-Library evaluations typically run 20–30 trials per instance, reducing the effective precomputation overhead to well under one second per trial.

Contrast with repair. These cost structures differ fundamentally. The repair operator is applied to every infeasible offspring; at $\alpha = 0.25$ under uniform initialization, essentially every offspring is infeasible, so repair runs on close to 100% of evaluations throughout the GA budget. Its total cost therefore scales linearly with offspring count T and problem size n. GF precomputation does not scale with T at all: regardless of how many offspring are evaluated, the p_j values are computed exactly once.

7.4 Limitations

- **Order-1 approximation:** The framework estimates per-item feasibility independently using order-1 schemas. For items with strong pairwise interactions, the order-1 approximation may over- or under-estimate the true per-item feasibility contribution. Extending to order-2 schemas is straightforward but increases computation by a factor of n.
- **Single-instance generalization:** Our experiments use a single instance per tightness level. Results on OR-Library benchmark instances will be presented in follow-up work.
- **Interaction with repair:** On well-structured standard MKP instances, repair effectively compensates for poor initialization and GF-biased initialization offers little advantage. The benefit is expected to emerge on adversarial or objective-free instances where repair is unavailable.

- **Soft penalty parity:** Under infeasibility penalty models, the initialization advantage does not consistently translate to improvements in final solution quality. The primary benefit is gen-0 feasibility and early convergence speed, which is most consequential under death penalty or when feasibility is difficult to recover through evolution.
- **ILS and hybrid parameters:** The ILS configuration ($r = 50$, $k = 5$) and the hybrid informed fraction are unoptimized. The experiments show that these extensions can substantially improve quality; they do not establish optimal settings.

8 Conclusion and Future Work

We have presented a generating function-informed initialization framework for genetic algorithms on the Multidimensional Knapsack Problem. The framework draws on the same algebraic tradition—encoding combinatorial structure as coefficients of formal power series—that Hardy and Ramanujan used to analyze integer partitions, and applies it to a new question: how much feasible space does each item's inclusion or exclusion preserve? By computing per-item feasibility densities from sub-generating functions and deriving per-bit sampling probabilities proportional to these densities, we obtain an initialization method that is principled, purely constraint-driven, and self-calibrating: it degenerates to uniform initialization exactly when no constraint structure is present.

The central empirical result has two parts. Under a death penalty fitness model at $\alpha = 0.25$, GF-biased initialization is the only method that functions—producing 49.8% initial feasibility versus 6.4% for Hill's [6] global-density method and 0% for uniform, then converging to 97.0% of the reference best while both baselines find nothing. Adding lightweight ILS post-processing raises this to 99.3%. Under soft penalty, GF-biased initialization consistently reaches Conv-90 fastest at $\alpha \geq 0.30$, with GF-biased converging faster than both baselines in the early phase. A quality-filtered seeding extension—generating $k \cdot P$ biased candidates and seeding with the top P feasible solutions by objective value—achieves 100% gen-0 feasibility and further reduces Conv-90 by 23% over standard GF-biased at $\alpha = 0.25$ (Section 6.10). A budget sensitivity analysis confirms that the initialization advantage is largest at low evaluation budgets: at 200 offspring, quality-filtered seeding outperforms uniform by 17.7 percentage points—a gap that would require approximately 3× the budget to close under uniform initialization (Section 6.10).

8.1 Future Work

1. **Adversarial instances (Part 2):** Generating MKP instances following the protocol of Hill et al. [8] with negative value-weight correlation to demonstrate regimes where repair-based approaches degrade while GF-biased initialization remains effective. Pisinger's hard knapsack instances [5], which exhibit strong item-weight correlations, represent an important additional test case where the order-1 approximation may be stressed.
2. **Constraint satisfaction:** Applying the framework to pure CSP instances where repair operators are undefined.
3. **OR-Library benchmarks:** Scaling to standard benchmark instances with n up to 500 and m up to 30. The standard practice of running 20–30 trials per instance makes the amortized precomputation cost per trial negligible.
4. **Higher-order schemas:** Extending feasibility density estimation to order-2 schemas to capture pairwise item interactions.
5. **Integration with repair:** Investigating whether GF-biased initialization improves the starting point for repair-based GAs on tight benchmark instances.

Acknowledgments

The author thanks Professor Raymond R. Hill (Air Force Institute of Technology) for detailed feedback on an inital draft of this paper Section 6.6.

References

[1] P. C. Chu and J. E. Beasley, *A Genetic Algorithm for the Multidimensional Knapsack Problem*, Journal of Heuristics, 4(1):63–86, 1998.

[2] J. E. Beasley, *OR-Library: Distributing Test Problems by Electronic Mail*, Journal of the Operational Research Society, 41(11):1069–1072, 1990.

[3] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, MI, 1975.

[4] H. S. Wilf, *generatingfunctionology*, 2nd edition, Academic Press, 1994.

[5] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*, Springer, Berlin, 2004.

[6] R. R. Hill, *A Monte Carlo Study of Genetic Algorithm Initial Population Generation Methods*, In *Proceedings of the 1999 Winter Simulation Conference*, eds. P. A. Farrington, H. B. Nembhard, D. T. Sturrock, and G. W. Evans, pp. 543–547, 1999.

[7] R. R. Hill and C. Hiremath, *Improving Genetic Algorithm Convergence Using Problem Structure and Domain Knowledge in Multidimensional Knapsack*

Problems, International Journal of Operational Research, 1(1–2):145–159, 2005.

[8] R. R. Hill, J. T. Moore, C. Hiremath, and Y. K. Cho, *Test Problem Generation of Binary Knapsack Problem Variants and the Implications of their Use*, International Journal of Operations and Quantitative Management, 18(2):105–128, 2012.

[9] G. H. Hardy and S. Ramanujan, *Asymptotic Formulae in Combinatory Analysis*, Proceedings of the London Mathematical Society, 2(17):75–115, 1918.